

EX NO: 01**BASICS SECURITY COMMANDS IN LINUX/UNIX****Date:****AIM:-**

To learn and run basic security commands in Linux.

Description: In this experiment, execute commands related to security basics and develop practical skills in Linux security.

1. User Management Commands**a.adduser / useradd**

- **Purpose:** Add new users to the system.
- **Syntax:** sudo adduser [username] or sudo useradd [username]

Example:

sudo adduser john

```
[egsp@localhost ~]$ sudo adduser john
```

```
[egsp@localhost ~]$ id john
```

```
uid=1006(john) gid=1009(john) groups=1009(john)
```

b. passwd

Purpose: Change user passwords.

Syntax: passwd [username]

Example:

sudo passwd john

```
[egsp@localhost ~]$ sudo passwd john
```

```
New password:*****
```

2. File Permissions and**Ownership chmod:**

Changes file permissions. It is used to control who can read, write, or execute files and directories.

Example usage: chmod [options] mode file

chmod +x filename

This command adds the execute (x) permission for the owner of the

file. Purpose: Modify file permissions.

Syntax: chmod [permissions] [file]

Example:

```
[egsp@localhost ~]$ touch jone.txt
```

```
[egsp@localhost ~]$ chmod 600 jone.txt
```

chown:

Changes the owner of a file or directory. It is often used to assign ownership to a specific user or group.

Example usage:

chown (Change Owner)

- **Purpose:** Change file owner and group.
- **Syntax:** chown [owner]:[group] [file]
- **Example:**

```
[egsp@localhost ~]$ sudo chown john:admin jone.txt
```

4. **chgrp:** Changes the group ownership of a file or directory. It helps in managing group permissions.

Example usage:

chgrp [options] new_group file

```
[egsp@localhost ~]$ sudo chown john jone.txt
```

4. **traceroute:** Traces the route taken by packets to a specified destination.

```
[egsp@localhost ~]$ traceroute google.com
```

```
traceroute to google.com (142.250.196.78), 30 hops max, 60 byte packets
```

```
1 _gateway (172.16.20.1) 0.213 ms 0.184 ms 0.173 ms
```

Result: Thus, the basic commands were run successfully on Linux.

EX NO: 2**NETWORKING COMMANDS IN LINUX****Date:**

Aim: To study and run different networking commands such as tcpdump, netstat, ifconfig, nslookup, and traceroute in Linux.

Description:**1. tcpdump:**

A command-line packet analyzer tool used to capture or filter TCP/IP packets.

Example:

```
[egsp@localhost ~]$ sudo tcpdump
```

```
12:42:07.746520 00:9e:1e:16:4e:f0 (oui Unknown) > 34:db:fd:77:e4:61 (oui Unknown),  
ethertype Unknown (0xa0a0), length 60:
```

```
0x0000: 0017 0101 0101 0101 0101 0101 0101 0101 .....
```

```
0x0010: 0101 0101 0101 0101 0101 0101 0101 0101 .....
```

```
0x0020: 0101 0101 0101 0101 0101 0101 0101 .....
```

2. netstat: Displays network connections, routing tables, and interface statistics.

```
[egsp@localhost ~]$ sudo netstat -tulnp
```

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State	PID/Program name
tcp	0	0	0.0.0.0:22	0.0.0.0:*	LISTEN	1207/sshd: /usr/sbi
tcp	0	0	127.0.0.54:53	0.0.0.0:*	LISTEN	955/systemd-resolve
tcp	0	0	0.0.0.0:5355	0.0.0.0:*	LISTEN	955/systemd-resolve
tcp	0	0	127.0.0.53:53	0.0.0.0:*	LISTEN	955/systemd-resolve
tcp6	0	0	:::22	:::*	LISTEN	1207/sshd: /usr/sbi
tcp6	0	0	:::9090	:::*	LISTEN	1/systemd
tcp6	0	0	:::5355	:::*	LISTEN	955/systemd-resolve
udp	0	0	127.0.0.54:53	0.0.0.0:*		955/systemd-resolve
udp	0	0	127.0.0.53:53	0.0.0.0:*		955/systemd-resolve
udp	0	0	127.0.0.1:323	0.0.0.0:*		1098/chronyd
udp	0	0	0.0.0.0:5355	0.0.0.0:*		955/systemd-resolve
udp6	0	0	:::1:323	:::*		1098/chronyd
udp6	0	0	:::5355	:::*		955/systemd-resolve

3. ifconfig: Configures network interfaces and views network parameters.

```
[egsp@localhost ~]$ ifconfig
```

```
enp1s0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500  
inet 172.16.22.188 netmask 255.255.252.0 broadcast 172.16.23.255  
inet6 fe80::c225:a5ff:fe96:6e24 prefixlen 64 scopeid 0x20<link>  
ether c0:25:a5:96:6e:24 txqueuelen 1000 (Ethernet)  
RX packets 466296 bytes 46230882 (44.0 MiB)
```

```
RX errors 0 dropped 440 overruns 0 frame 0
TX packets 1836 bytes 290024 (283.2 KiB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
inet 127.0.0.1 netmask 255.0.0.0
inet6 ::1 prefixlen 128 scopeid 0x10<host>
loop txqueuelen 1000 (Local Loopback)
RX packets 81 bytes 6308 (6.1 KiB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 81 bytes 6308 (6.1 KiB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

4. **nslookup:** Queries DNS servers to obtain DNS-related information.

```
[legsp@localhost ~]$ nslookup google.com
```

```
Server:      127.0.0.53
Address:     127.0.0.53#53
```

```
Non-authoritative answer:
```

```
Name: google.com
Address: 142.250.182.46
Name: google.com
Address: 2404:6800:4007:81e::200e
```

Result: The password policy was successfully implemented, and user accounts were managed effectively in the Linux environment.

EX NO: 3

PASSWORD POLICY AND USER ACCOUNT MANAGEMENT

Date:

Aim:

To implement password policy and manage user accounts effectively in a Linux environment.

Description:

This experiment combines the implementation of password policy and user account management to enhance security in Linux systems. It covers configuring password policies such as length, complexity requirements, and expiration periods, along with creating, modifying, and deleting user accounts

Password Policy Implementation:

Description: Implement password policies to enforce security standards for user passwords.

Steps:

Set a minimum password length.

Configure password complexity requirements.

Implement password expiration periods.

Example:

```
[egsp@localhost]$ sudo chage -M 90 egspec  
[egsp@localhost lakshman]$ sudo chage -m 7 egspec  
[egsp@localhost lakshman]$ sudo chage -d 2025-04-07 egspec
```

User Account Management:

Description: Manage user accounts to control access to the system securely.

Steps:

Create new user accounts.

Modify user account properties.

Delete user accounts.

```
[egsp@localhost]$ sudo useradd john
```

```
[sudo] password for egsp:
```

```
useradd: user 'john' already exists
```

```
[egsp@localhost]$ sudo usermod [options] egspec
```

```
Usage: usermod [options] LOGIN
```

Options:

- a, --append append the user to the supplemental GROUPS mentioned by the -G option without removing the user from other groups
- b, --badname allow bad names
- c, --comment COMMENT new value of the GECOS field
- d, --home HOME_DIR new home directory for the user account
- e, --expiredate EXPIRE_DATE set account expiration date to EXPIRE_DATE
- f, --inactive INACTIVE set password inactive after expiration to INACTIVE
- g, --gid GROUP force use GROUP as new primary group
- G, --groups GROUPS new list of supplementary GROUPS
- h, --help display this help message and exit
- l, --login NEW_LOGIN new value of the login name
- L, --lock lock the user account
- m, --move-home move contents of the home directory to the

```
[egsp@localhost]$ sudo userdel [options] egspec
```

```
Usage: userdel [options] LOGIN
```

Options:

- f, --force force some actions that would fail otherwise e.g. removal of user still logged in or files, even if not owned by the user
- h, --help display this help message and exit
- r, --remove remove home directory and mail spool
- R, --root CHROOT_DIR directory to chroot into
- P, --prefix PREFIX_DIR prefix directory where are located the /etc/* files
- Z, --selinux-user remove any SELinux user mapping for the user

Password Policy Implementation:

Set a minimum password length using the **passwd** command.

Example: passwd --minlen=8

This command sets the minimum password length to 8 characters.

Configure password complexity requirements in the /etc/pam.d/common-password file.

Example: Update /etc/pam.d/common-password to enforce complexity rules.

Modify the password requisite pam_pwquality.so line to specify complexity requirements.

Implement password expiration periods using the chage command.

Example: chage -M 90 username

This command sets the maximum password age for the user username to 90 days.

User Account Management:

Create a new user account using the **useradd** command.

Example: useradd username

This command creates a new user account named username.

Modify user account properties using the **usermod** command.

Example: usermod -aG groupname username

This command adds the user username to the supplementary group groupname.

Delete a user account using the **userdel** command.

Example: userdel username

This command deletes the user account username.

Result:

The password policy was successfully implemented, and user accounts were managed effectively in the Linux environment.

EXNO:4 ROLE BASED ACCESS CONTROL (RBAC) SETUP IN LINUX**Date:****Aim:**

To implement Role-based Access Control (RBAC) for managing user privileges effectively in a Linux environment.

Description:

Role-based Access Control (RBAC) is a security paradigm where access rights are granted to users based on their roles within an organization. This experiment focuses on setting up RBAC in a Linux environment to define roles, assign permissions, and manage user access more efficiently.

Define Roles with Specific Permissions:

Identify the different roles within the organization and define the specific permissions associated with each role.

Example :**Define roles such as admin, developer, and operator.**

```
[egsp@localhost]$ sudo usermod -aG admin egspec  
[egsp@localhost]$ sudo usermod -aG guest egspec  
[egsp@localhost]$ sudo usermod -aG user egspec  
[egsp@localhost]$ sudo usermod -aG developer egspec
```

The admin role may have permissions to install software and configure system settings, while the developer role may have permissions to access development tools and directories.

Define roles:**sudo groupadd admin****sudo groupadd developer****sudo groupadd operator****Assign permissions:**

```
sudo chmod -R 750 /var/www/html # Allow admin group full access to web directory
```

```
sudo chown -R root:admin /var/www/html
```

```
sudo chmod -R 750 /opt/dev_tools # Allow developer group full access to development tools
```

```
sudo chown -R root:developer /opt/dev_tools
```

```
sudo chmod -R 750 /var/log/nginx # Allow operator group access to NGINX logs
```

```
sudo chown -R root:operator /var/log/nginx
```

Example:

```
[egsp@localhost ~]$ sudo chmod -R 750/var/www/html
[egsp@localhost ~]$ sudo chown -R root:admin/var/www/html
```

The permissions are set accordingly, allowing the specified groups access to their respective directories.

Assign Roles to Users or Groups:

Once roles are defined, assign them to individual users or groups as per their job functions.

Once roles are defined, assign them to individual users or groups as per their job functions.

Example:

```
[egsp@localhost ~]$ sudo usermod -aG admin sysadmin
[egsp@localhost ~]$ V sudo usermod -aG developer alice,bob
[egsp@localhost ~]$ sudo usermod -aG operator operator1,operator2
```

Assign roles:

Users `admin_user`, `dev_user1`, `dev_user2`, `op_user1`, and `op_user2` are added to their respective groups, granting them access to the associated permis

Implement sudo for Temporary Privilege Escalation:

Utilize sudo to grant temporary elevated privileges to users for specific tasks that require administrative permissions.

Example:

```
[egsp@localhost ~]$ sudo chown :admin /path/to/admin_files
[egsp@localhost ~]$ sudo chmod 770 /path/to/admin_files
[egsp@localhost ~]$ sudo chown :developer /path/to/dev_tools
[egsp@localhost ~]$ sudo chmod 770 /path/to/dev_tools
[egsp@localhost ~]$ sudo chown :operator /path/to/sys_logs~]$
[egsp@localhost ~]$ sudo chmod 740 /path/to/sys_logs
```

Configure sudo:

sudo visudo

Add the following line to the sudoers file:

```
%admin ALL=(ALL) ALL
```

Users in the admin group can now execute any command as root using sudo, providing temporary privilege escalation.

Configure SELinux Policies for Access Control:

SELinux provides additional security measures by enforcing mandatory access control policies. Configure SELinux policies to restrict access based on roles and permissions.

Example:

```
[egsp@localhost~]$ sudo semanage login -a -s user_u admin_user
[egsp@localhost~]$ sudo semanage login -a -s user_u dev_user1
[egsp@localhost~]$ sudo semanage login -a -s user_u dev_user2
[egsp@localhost~]$ sudo semanage login -a -s user_u op_user1
[egsp@localhost~]$ sudo semanage login -a -s user_u op_user2
```

Configure SELinux policies:

```
sudo setsebool -P httpd_read_user_content 1 # Allow Apache to read user content
```

```
sudo semanage fcontext -a -t httpd_sys_content_t "/var/www/html(/.*)?" # Allow Apache access to web directory
```

```
sudo semanage fcontext -a -t httpd_sys_content_t "/opt/dev_tools(/.*)?" # Allow Apache access to development tools
```

```
sudo semanage fcontext -a -t httpd_sys_content_t "/var/log/nginx(/.*)?" # Allow NGINX access to logs
```

SELinux policies are configured to allow Apache and NGINX access to the specified directories according to RBAC.

Result:

The Role-based Access Control (RBAC) setup in Linux was successfully implemented

EXNO:5**AUDITING AND LOGGING CONFIGURATION IN LINUX****Date:****Aim:**

To configure auditing and logging mechanisms to monitor system activity in a Linux environment.

Description:

Auditing and logging are essential components of system security, allowing administrators to track and review system events for security analysis and compliance. This experiment focuses on configuring auditing and logging mechanisms in Linux to monitor system activity effectively.

Configure Auditing with auditd:

Use the auditd service to configure auditing rules and monitor system activity.

Example:

```
[egsp@localhost ~]$ sudo ausearch -k etc_access
```

```
[sudo] password for egsp:
```

```
time->Fri Mar 28 10:28:21 2025
```

```
type=CONFIG_CHANGE msg=audit(1743137901.336:294): auid=1000 ses=5 subj=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 op=add_rule key="etc_access" list=4 res=1
```

```
time->Fri Mar 28 10:55:04 2025
```

```
type=CONFIG_CHANGE msg=audit(1743139504.982:1195): auid=4294967295 ses=4294967295 subj=system_u:system_r:unconfined_service_t:s0 op=remove_rule key="etc_access" list=4 res=1
```

```
time->Fri Mar 28 10:56:19 2025
```

```
type=CONFIG_CHANGE msg=audit(1743139579.803:1243): auid=1000 ses=5 subj=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 op=add_rule key="etc_access" list=4 res=1
```

```
time->Fri Mar 28 12:05:26 2025
```

```
type=CONFIG_CHANGE msg=audit(1743143726.780:1826): auid=4294967295 ses=4294967295 subj=system_u:system_r:unconfined_service_t:s0 op=remove_rule key="etc_access" list=4 res=1
```

```
-----
```

```
time->Mon Apr 7 12:11:09 2025
```

```
type=CONFIG_CHANGE msg=audit(1744008069.825:812): auid=1000 ses=6 subj=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 op=add_rule key="etc_access" list=4 res=1
```

Output:

Watched file /etc is associated with audit event type 'modify' and key 'etc_access'

Analyze System Logs:

Utilize system log files to review audit events and monitor system activity. Example:

```
[egsp@localhost ~]$ sudo less /var/log/audit/audit.log
```

Implement Log Rotation and Retention Policies:

Configure log rotation and retention policies to manage log files efficiently and ensure compliance with storage requirements.

Example:

```
[egsp@localhost ~]$ sudo nano /etc/logrotate.d/audit
```

Add the following configuration:

```
/var/log/audit/audit.log {  
    weekly  
    rotate 4    delaycompress  
    missingok  
    notifempty
```

```
}
```

the configuration will be saved.

Result:

The auditing and logging mechanisms in the Linux environment were successfully configure

EXNO: 6 PACKAGE MANAGEMENT AND SECURITY IN LINUX UPDATES**Date:****Aim:**

To Effectively manage software packages and ensure timely security updates on your Linux system.

Description:

package management tools to keep your system up-to-date and secure Understanding package management tools and configuring automatic updates for security patches are critical tasks for system administrators and users alike.

Procedure:**Determining the Package Manager:**

The first step is to identify the package manager used by your Linux distribution. Common package managers include apt for Debian/Ubuntu-based distributions and yum for Red Hat/CentOS-based distributions. Use the appropriate command to determine the package manager installed on your system:

For Debian/Ubuntu: `dpkg --get-architecture | grep apt`

For Red Hat/CentOS: `rpm -q yum`

Package Management Commands:

Once you have identified the package manager, practice using the following commands to manage software packages:

For Debian/Ubuntu: apt list**Example:**

```
$ apt list
```

```
Listing... Done
```

```
package1/now 1.0.0-1 amd64 [installed,upgradable to: 1.1.0-1]
```

```
package2/now 2.0.0-1 amd64 [installed]
```

For Red Hat/CentOS: `yum list`

Example:

```
$ yum list
```

```
Installed Packages
```

package1.x86_64	1.0.0-1	@repository
package2.x86_64	2.0.0-1	@repository

Available Packages

package3.x86_64	3.0.0-1	repository
-----------------	---------	------------

Installing a Specific Package:

For Debian/Ubuntu: apt install package name

Example:

```
$ apt install package3
```

```
Reading package lists... Done
```

```
Building dependency tree
```

```
Reading state information... Done
```

The following NEW packages will be installed:

```
package3
```

```
0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
```

```
Need to get 100 kB of archives.
```

```
After this operation, 200 kB of additional disk space will be used.
```

For Red Hat/CentOS: yum install package_name

Example:

```
$ yum install package4
```

```
Dependencies resolved.
```

```
=====
=====
```

Package	Arch	Version	Repository	Size
---------	------	---------	------------	------

```
=====
=====
```

Installing:

package4	x86_64	4.0.0-1	repository	200 kB
----------	--------	---------	------------	--------

Updating the System and Installing Security Patches:

For Debian/Ubuntu: apt update && apt upgrade

```
$ apt update && apt upgrade
```


...

10 packages can be upgraded. Run 'apt list --upgradable' to see them.

...

For Red Hat/CentOS: yum update

Example:

\$ yum update

...

Security: package5-1.0.1-1.x86_64 is an installed security update

...

Removing a Package:

For Debian/Ubuntu: apt remove package_name

Example:

\$ apt remove package2

Reading package lists... Done

Building dependency tree

Reading state information... Done

The following packages will be REMOVED:

package2

0 upgraded, 0 newly installed, 1 to remove and 0 not upgraded.

After this operation, 50 kB disk space will be freed.

For Red Hat/CentOS: yum remove package_name

Example:

\$ yum remove package3

Dependencies resolved.

```
=====
```

```
=====
```

Package	Arch	Version	Repository	Size
---------	------	---------	------------	------

```
=====
```

```
=====
```

Removing:

package3 x86_64 3.0.0-1 @repository 50 kB

Configuring Automatic Updates:

To ensure timely installation of security patches, configure automatic updates for security packages using your distribution's tools or package manager settings. The procedure may vary depending on your Linux distribution. Consult the documentation or online resources specific to your distribution for detailed instructions

Example

configure automatic updates on a Debian-based system using the unattended-upgrades package.

Install the unattended-upgrades Package:

```
$ sudo apt-get update
```

```
$ sudo apt-get install unattended-upgrades
```

This command updates the package lists and installs the unattended-upgrades package, which allows automatic installation of security updates.

Configure Automatic Updates:

Edit the configuration file `/etc/apt/apt.conf.d/50unattended-upgrades` to specify which packages should be automatically upgraded.

```
$ sudo nano /etc/apt/apt.conf.d/50unattended-upgrades
```

Within the configuration file, you can specify the following options:

Unattended-Upgrade::Allowed-Origins: Specify the sources from which updates should be automatically installed.

Unattended-Upgrade::Package-Blacklist: Specify packages that should not be automatically upgraded.

Unattended-Upgrade::Automatic-Reboot: Specify whether the system should automatically reboot after installing updates.

Enable Automatic Updates:

Edit the configuration file `/etc/apt/apt.conf.d/20auto-upgrades` to enable automatic updates.

```
$ sudo nano /etc/apt/apt.conf.d/20auto-upgrades
```

Set the APT::Periodic::Update-Package-Lists and APT::Periodic::Unattended-Upgrade options to 1 to enable automatic updates.

```
APT::Periodic::Update-Package-Lists "1";
```

```
APT::Periodic::Unattended-Upgrade "1";
```

Verify Configuration:

You can verify that automatic updates are configured correctly by checking the contents of the configuration files and ensuring that the desired options are set.

```
$ cat /etc/apt/apt.conf.d/50unattended-upgrades
```

```
$ cat /etc/apt/apt.conf.d/20auto-upgrades
```

Example Output:

After completing the configuration steps, your /etc/apt/apt.conf.d/50unattended-upgrades file may look like this:

```
// Automatically upgrade packages from these (origin:archive) pairs
```

```
Unattended-Upgrade::Allowed-Origins {
```

```
    "${distro_id}:${distro_codename}-security";
```

```
// "${distro_id}:${distro_codename}-updates";
```

```
// "${distro_id}:${distro_codename}-proposed";
```

```
// "${distro_id}:${distro_codename}-backports";
```

```
};
```

```
// List of packages to not update (regexp are supported)
```

```
Unattended-Upgrade::Package-Blacklist {
```

```
// "vim";
```

```
// "libc6";
```

```
// "libc6-dev";
```

```
// "libc6-i686";
```

```
};
```

```
// Automatically reboot *WITHOUT CONFIRMATION* if a
// the file /var/run/reboot-required is found after the upgrade
Unattended-Upgrade::Automatic-Reboot "true";
```

And your /etc/apt/apt.conf.d/20auto-upgrades file may look like this:

```
APT::Periodic::Update-Package-Lists "1";
APT::Periodic::Unattended-Upgrade "1";
```

Execution:

Determine the package manager installed on your Linux system using the appropriate command mentioned in step 1 of the procedure.

Determine the Package Manager:

Use the appropriate command to determine the package manager installed on your Linux system:

For Debian/Ubuntu: `dpkg --get-selections | grep apt`

For Red Hat/CentOS: `rpm -q yum`

```
$ dpkg --get-selections | grep apt
```

```
ii apt          1.6.5ubuntu0.5 amd64 commandline package manager
```

Package Management Commands:

Practice using the package management commands to perform various tasks such as listing available packages, installing, updating, and removing packages.

```
$ apt list
```

```
Listing... Done
```

```
package1/now 1.0.0-1 amd64 [installed,upgradable to: 1.1.0-1]
```

```
package2/now 2.0.0-1 amd64 [installed]
```

explore the options for configuring automatic updates for security packages on your Linux distribution. Follow the recommended procedures to enable automatic updates and verify that the settings are applied

correctly.

Result:

The package management and security procedures on the Linux system were executed successfully.

EXNO:7 VULNERABILITY SCANNING AND ASSESSMENT IN LINUX**Date:****Aim:**

To conduct vulnerability scanning and assessment on a Linux system to identify potential security vulnerabilities.

Description:

Vulnerability scanning and assessment are crucial components of proactive security measures. By using specialized tools, you can systematically identify weaknesses in your system's configuration, software, or policies that could be exploited by attackers. This experiment introduces vulnerability scanning tools and guides you through the process of identifying and prioritizing security vulnerabilities on your Linux system

Procedure:

Choose a Suitable Vulnerability Scanner:

Research and select a vulnerability scanning tool compatible with your Linux distribution. Common vulnerability scanners include OpenVAS, Nessus, and Nikto. Consider factors such as features, ease of use, and compatibility with your system.

Install and Configure the Chosen Scanner:

Follow the documentation provided by the chosen vulnerability scanner to install and configure it on your Linux system. Depending on the scanner, this may involve downloading and installing the software package, setting up necessary dependencies, and configuring scanning parameters.

Run a Vulnerability Scan:

Initiate a vulnerability scan of your Linux system using the configured scanner. Configure the scan parameters according to your requirements, such as the scope of the scan (e.g., network, specific hosts) and the types of vulnerabilities to scan for (e.g., software vulnerabilities, misconfigurations).

Analyze the Scan Results:

After the vulnerability scan completes, analyze the generated report to identify potential security vulnerabilities and weaknesses in your system. Pay attention to severity ratings, additional information about potential impacts, and recommended remediation steps provided in the report.

Prioritize and Remediate Vulnerabilities:

Prioritize identified vulnerabilities based on severity ratings and potential impacts on your system's security. Develop a remediation plan to address these vulnerabilities, which may involve applying software patches, implementing configuration changes, or deploying additional security controls.

Execution:

Determine the appropriate vulnerability scanner for your Linux distribution based on compatibility and features.

Install and Configure OpenVAS:

```
$ sudo apt update
```

```
$ sudo apt install openvas
```

This command updates the package lists and installs the OpenVAS package on your Ubuntu system.

Run a Vulnerability Scan:

```
$ sudo openvas-check-setup --v9
```

This command checks the setup of OpenVAS and provides detailed information about the status of various components and services.

Example:

```
[egsp@localhost ~]$ sudo openvas-check-setup --v9
Checking OpenVAS Scanner ...
OK: OpenVAS Scanner is present in version 5.1.1.
OK: redis-server is present in version v=5.0.7.
OK: scanner (kb_location setting) is configured properly using the redis-server socket:
/tmp/redis.sock
OK: redis-server is running and listening on socket: /tmp/redis.sock.
OK: redis-server configuration is OK and redis-server is running.
OK: NVT collection in /var/lib/openvas/plugins contains 56621 NVTs.
WARNING: Signature checking of NVTs is not enabled in OpenVAS Scanner.
SUGGEST: Enable signature checking (see http://www.openvas.org/trusted-nvts.html).
```

The output provides information about the status of various components of OpenVAS and suggests improvements, such as enabling signature checking for Network Vulnerability Tests (NVTs).

Prioritize and Remediate Vulnerabilities

Based on the scan results, prioritize and remediate identified vulnerabilities according to their severity ratings. Develop a remediation plan, which may involve applying software patches, implementing configuration changes, or deploying additional security controls.

:

Result:

Vulnerability scanning and assessment on the Linux system using OpenVAS was successfully conducted

EXNO:8**SYSTEM HARDENING TECHNIQUES****Date:****Aim:**

To implement system hardening techniques to strengthen the overall security of a Linux system.

Description:

System hardening involves implementing various security measures to minimize potential attack surfaces and enhance the resilience of a system against cyber threats. This experiment focuses on applying system hardening techniques to mitigate security risks on a Linux system. You will learn how to disable unnecessary services, strengthen user account and password policies, secure sudo access, and restrict access to sensitive files and directories.

Procedure:

Identify Unnecessary Services:

Use tools such as `chkconfig` or `systemctl` to identify unnecessary services running on the system. Review the list of services and determine which ones can be safely disabled to reduce potential attack vectors.

Disable Unused Services:

Disable unnecessary services to minimize the attack surface of the system. Use the appropriate commands (`chkconfig`, `systemctl`, etc.) to stop and disable services that are not required for system functionality.

Strengthen User Account and Password Policies:

Review and enforce strong user account and password policies to prevent unauthorized access to the system. This may involve setting password complexity requirements, enforcing password expiration, and limiting user access privileges.

Secure Sudo Access:

Configure sudo access with the principle of least privilege, granting only necessary permissions to specific users or groups. Edit the `/etc/sudoers` file using the `visudo` command to specify sudo access rules and ensure that sudo access is restricted to authorized users.

Restrict Access to Sensitive Files and Directories:

Adjust permissions on sensitive files and directories to restrict access only to authorized users or groups.

Use the chmod and chown commands to modify file and directory permissions as needed, ensuring that sensitive information is protected from unauthorized access.

Execution:

Identify Unnecessary Services:

Use systemctl command to list running services:

```
[egsp@localhost ~]$ systemctl list-units --type=service --state=running
```

UNIT	LOAD	ACTIVE	SUB	DESCRIPTION
accounts-daemon.service	loaded	active	running	Accounts Service
acpid.service	loaded	active	running	ACPI event daemon
bluetooth.service	loaded	active	running	Bluetooth service

Identify services that are not essential for system functionality.

Disable Unused Services:

Disable unnecessary services using systemctl:

```
$ sudo nano /etc/login.defs
```

Replace <service_name> with the name of the service to be disabled.

Strengthen User Account and Password Policies:

Edit the /etc/login.defs file to enforce password policies:

```
[egsp@localhost ~]$ sudo nano /etc/login.defs
```

```
PASS_MAX_DAYS 90
PASS_MIN_DAYS 7
PASS_WARN_AGE 7
```

Configure parameters such as `PASS_MAX_DAYS`, `PASS_MIN_DAYS`, `PASS_WARN_AGE` to enforce password expiration, minimum and maximum password age, and warning period.

Secure Sudo Access:

Edit the `/etc/sudoers` file using `visudo` command:

```
[egsp@localhost ~]$ sudo visudo
```

Add the following line to grant sudo access to a user:

```
username ALL=(ALL) ALL
```

Replace username with the actual username.

Restrict Access to Sensitive Files and Directories:

Use `chmod` and `chown` commands to modify permissions on sensitive files and directories

```
$ sudo chmod 600 /path/to/sensitive_file
```

```
$ sudo chown root:root /path/to/sensitive_directory
```

Replace `/path/to/sensitive_file` and `/path/to/sensitive_directory` with the actual paths.

Example :

Identify Unnecessary Services:

```
[egsp@localhost ~]$ systemctl list-units --type=service --state=running
```

UNIT	LOAD	ACTIVE	SUB	DESCRIPTION
accounts-daemon.service	loaded	active	running	Accounts Service
acpid.service	loaded	active	running	ACPI event daemon

...

Disable Unused Services:

```
$ sudo systemctl stop bluetooth.service
```

```
$ sudo systemctl disable bluetooth.service
```

Strengthen User Account and Password Policies:

After editing /etc/login.defs, the changes will be applied to the system. No specific output is generated.

Secure Sudo Access:

After adding the user to the sudoers file, the user will have sudo access. No specific output is generated.

Restrict Access to Sensitive Files and Directories:

```
[egsp@localhost ~]$ sudo chmod 600 /etc/shadow
```

```
[egsp@localhost ~]$ sudo chown root:root /etc/ssh
```

Result:

The system hardening techniques were successfully implemented on the Linux system.

EXNO:9**FIREWALL CONFIGURATION LINUX****Date:****Aim:**

To configure a firewall on a Linux system to control incoming and outgoing network traffic.

Description:

Firewalls play a crucial role in network security by acting as a barrier between a trusted internal network and untrusted external networks, such as the internet. By configuring a firewall, you can control which network traffic is allowed or blocked, thereby protecting your system from unauthorized access and potential security threats. This experiment introduces firewall configuration techniques on Linux systems using tools like iptables or ufw.

Procedure:

Choose a Firewall Tool:

Research and choose a firewall tool suitable for your Linux distribution. Common firewall tools for Linux include iptables (traditional) and ufw (simplified interface for iptables).

Install the Firewall Tool:

If the chosen firewall tool is not already installed on your system, install it using your distribution's package manager.

Configure Firewall Rules:

Define firewall rules to control incoming and outgoing network traffic. Configure rules to allow or block traffic based on criteria such as source/destination IP addresses, ports, and protocols.

Test Firewall Rules:

Test the configured firewall rules to ensure they are working as expected. Verify that allowed traffic is permitted while blocked traffic is denied.

Persist Firewall Configuration:

Persist the firewall configuration so that it is applied automatically upon system startup or firewall service restarts.

Execution:**1. Choose a Firewall Tool:**

For this example, we'll use `ufw` (Uncomplicated Firewall), which provides a simplified interface for managing `iptables` firewall rules.

2. Install `ufw`:

```
...
```

```
$ sudo apt update
```

```
$ sudo apt install ufw
```

```
...
```

3. Configure Firewall Rules:

- Allow SSH (port 22) traffic:

```
...
```

```
$ sudo ufw allow ssh
```

```
...
```

- Allow HTTP (port 80) traffic:

```
...
```

```
$ sudo ufw allow http
```

```
...
```

- Allow HTTPS (port 443) traffic:

```
...
```

```
$ sudo ufw allow https
```

```
...
```

- Block incoming traffic from a specific IP address:

```
...
```

```
$ sudo ufw deny from <ip_address
```

4. Test Firewall Rules:

- List current firewall rules:

```
'''
```

```
$ sudo ufw status
```

```
'''
```

Example Output:

```
'''
```

Status: active

To	Action	From
---	-----	-----
OpenSSH	ALLOW	Anywhere
80	ALLOW	Anywhere
443	ALLOW	Anywhere
'''		

- Attempt to establish connections to verify that allowed traffic is permitted and blocked traffic is denied.

5. Persist Firewall Configuration:

Enable `ufw` to start on system boot:

```
'''
```

```
$ sudo ufw enable
```

```
'''
```

****Example Output:****

```
'''
```

Command may disrupt existing ssh connections. Proceed with operation (y|n)? y

Firewall is active and enabled on system startup

```

Example Output:

- Testing Firewall Rules:

After running `sudo ufw status`, you should see a list of configured rules similar to the following:

```

Status: active

To	Action	From
---	-----	----
OpenSSH	ALLOW	Anywhere
80	ALLOW	Anywhere
443	ALLOW	Anywhere

```

This output confirms that SSH, HTTP, and HTTPS traffic is allowed, while all other incoming traffic is blocked by default.

### Result:

The configuration of the firewall using ufw on the Linux system has been completed successfully.



**EXNO:10                      IMPLEMENTING FILE SYSTEM ENCRYPTION****Date:****Aim:**

To encrypt sensitive data on a Linux system to protect it from unauthorized access.

**Description:**

File system encryption is a critical security measure to safeguard sensitive data stored on disk drives. By encrypting the file system, you can ensure that even if an attacker gains access to the physical storage media, they won't be able to read the data without the encryption key. This experiment introduces file system encryption techniques on Linux systems using tools like LUKS (Linux Unified Key Setup) or eCryptfs.

**Procedure:****1. Choose a File System Encryption Tool:**

Research and choose a file system encryption tool suitable for your Linux distribution. Common encryption tools for Linux include LUKS and eCryptfs.

**2. Prepare the Disk Partition:**

Partition the disk where the sensitive data will be stored. Create a dedicated partition for the encrypted file system.

**3. Encrypt the File System:**

Use the chosen encryption tool to encrypt the disk partition. Configure encryption parameters such as encryption algorithm, key size, and passphrase.

**4. Mount the Encrypted File System:**

Mount the encrypted file system to make it accessible for storing and retrieving data. Ensure that the file system is properly mounted upon system boot.

**5. Test Encryption and Access Control:**

Test the encryption setup by writing data to the encrypted file system and verifying that it is encrypted at rest. Attempt to access the encrypted data without the encryption key to ensure that it remains protected.

**Execution:****1. Choose a File System Encryption Tool:**

For this example, we'll use LUKS (Linux Unified Key Setup) for disk encryption.

## 2. Prepare the Disk Partition:

Partition the disk using a tool like `fdisk` or `parted`:

...

```
$ sudo fdisk /dev/sdb
```

...

Create a new partition of type Linux (83) and write changes to the disk.

## 3. Encrypt the File System:

Format the partition with LUKS encryption:

...

```
$ sudo cryptsetup luksFormat /dev/sdb1
```

...

Enter a passphrase to encrypt the partition.

## 4. Mount the Encrypted File System

Open the encrypted partition and map it to a device:

...

```
$ sudo cryptsetup luksOpen /dev/sdb1 encrypted_partition
```

...

Create a file system on the mapped device:

...

```
$ sudo mkfs.ext4 /dev/mapper/encrypted_partition
```

...

Create a mount point and mount the encrypted file system:

...

```
$ sudo mkdir /mnt/encrypted_data
```

```
$ sudo mount /dev/mapper/encrypted_partition /mnt/encrypted_data
```

...

## 5. Test Encryption and Access Control:

Write test data to the encrypted file system:

...

```
$ echo "This is a test file" | sudo tee /mnt/encrypted_data/test.txt
```

...

Verify that the data is encrypted:

...

```
$ sudo hexdump -C /dev/sdb1
```

...

Attempt to access the encrypted data without providing the passphrase:

...

```
$ sudo umount /mnt/encrypted_data
```

```
$ sudo cryptsetup luksClose encrypted_partition
```

...

Try to mount the encrypted partition without providing the passphrase:

...

```
$ sudo cryptsetup luksOpen /dev/sdb1 encrypted_partition
```

...

**Example Output:****- Encrypt the File System:**

After running ``sudo cryptsetup luksFormat /dev/sdb1``, you will be prompted to enter a passphrase. Once entered, the partition will be encrypted using LUKS.

**- Mount the Encrypted File System:**

After running the mount commands, the encrypted file system will be mounted at ``/mnt/encrypted_data`` and ready for use.

**- Test Encryption and Access Control:**

After writing test data to the encrypted file system and attempting to access it without providing the passphrase, you should see that the data remains encrypted and inaccessible without the encryption key.

**Result:**

The implementation of file system encryption on a Linux system has been completed successfully.

